Software Requirements Specification

for

LAN-Based Secure Chatting Application

Version 1.0

TE Computer Engineering Mini Project

Prepared by

Aryan Patil — Networking / Java Server & Client

Malhar Nimbalkar — DBMS / Backend

Amogh Sirsirkar —PDS / UI

Date: September 6, 2026

Course: Software Engineering Mini Project

Revision History

Version	Date	Description	Author
0.1	Week 4	Initial draft based on Phase 1 proposal	All members
1.0	Prior draft	Complete SRS for review and submission	All members
1.1	Current	Requirements Consistency Audit performed (Section 20); 14 findings resolved — corrected traceability matrix, added FR-48/FR-49/NFR-21, standardized component naming, reconciled deployment vs. operating environment, reworded SEC-05 and NFR-12, added acceptance criteria and test cases	All members

Table of Contents:

Sr. No.:	Contents:	Page No.:
1	Introduction	3
2	Overall Description	5
3	System Architecture	6
4	External Interface Requirements	8
5	Functional Requirements	9
6	Non-Functional Requirements	11
7	Database Requirements	13
8	Security Requirements	14
9	Data Science / PDS Requirements	15
10	Use Cases	16
11	System Models	19
12	Testing Requirements	21
13	Risks and Mitigations	22
14	Representative Test Cases	23
15	Deployment Model	24
16	Explicit Project Limitations	24
17	Future Enhancements	24
18	Appendices	24

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for Version 1.0 of the LAN-Based Secure Chatting Application, a self-hosted communication platform intended for use within a single trusted Local Area Network (LAN). This document is intended for the project team, the course instructor/evaluator, and any future developer who extends the system. It specifies what the system shall do, not how the underlying code is organized, except where a design or implementation constraint is explicitly known and stated as such.

1.2 Scope

The system is a Java client/server chat application that allows users on the same LAN to register, authenticate, exchange one-to-one and group messages in near real time, share files, and search their persistent chat history stored in PostgreSQL. A supporting Python-based Probability and Data Science (PDS) module screens outgoing/incoming messages for spam or potentially misleading patterns using Logistic Regression, and a Linear Regression component estimates a measurable system metric (e.g., message delivery latency) for evaluation purposes.

The product is an academic mini-project. It is deliberately scoped to a small, explainable, LAN-only system and is not intended to compete with, or replace, Internet-scale platforms such as Signal, WhatsApp, or Telegram.

In Scope

- Self-hosted Java server and Java client operating on a single LAN.
- User registration, authentication, and online/offline presence.
- One-to-one real-time messaging with persistence in PostgreSQL.
- Persistent chat history in PostgreSQL.
- Hashed credential storage.
- A Python PDS module providing Logistic Regression-based spam/misleading-content screening.
- A Linear Regression component estimating a supporting system metric, evaluated with MAE/RMSE.

Out of Scope

- Internet-scale or multi-LAN/federated messaging.
- Voice calling, video calling, message reactions, stories, or cloud backup.
- Mobile applications; the client is a Java desktop application.
- Full end-to-end encryption.
- An AI chatbot, blockchain, biometric login, or any deep-learning/NLP model beyond the specified classical Logistic/Linear Regression.
- Automated fact-checking or truth verification of message content.
- An administrator role (not currently planned; see Section 2.3).

- Self-service password/account recovery (e.g., a “forgot password” e-mail flow) — not supported in Release 1.0; this is a deliberate scope decision, not an oversight, and is listed as a future enhancement in Section 19.

1.3 Intended Audience

- Project team members implementing the client, server, database, and UI.
- The course instructor / evaluator assessing the project against Software Engineering practice.
- Any future contributor extending the system beyond Release 1.0.

1.4 Product Overview

The LAN-Based Secure Chatting Application follows a client-server architecture. Multiple Java clients connect over TCP/IP to a single Java application server, which authenticates users, routes one-to-one messages, and persists conversations and messages to a PostgreSQL database. The system is intended for classrooms, computer laboratories, office LANs, and other small, trusted network environments where Internet-scale messaging infrastructure is unnecessary.

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
LAN	Local Area Network — the trusted network boundary within which this system operates.
PDS	Probability and Data Science — the academic subject and, in this document, the Python module implementing it.
TLS	Transport Layer Security — the encrypted transport mechanism proposed for client-server communication.
CRUD	Create, Read, Update, Delete — the basic set of persistence operations.
FR / NFR	Functional Requirement / Non-Functional Requirement.
MAE / RMSE	Mean Absolute Error / Root Mean Squared Error — evaluation metrics for the Linear Regression component.
Logistic Regression	A classical statistical classification algorithm producing a probability of class membership; not a deep-learning or generative-AI technique.
Session	The authenticated state established for a client connection after successful login, held until logout or disconnection.
ClientHandler	A conceptual server-side component responsible for one connected client's communication.

1.6 References

- Project Phase 1 Document: "LAN-Based Secure Chatting Application — A self-hosted local communication platform with secure messaging, PostgreSQL persistence, and lightweight data-science modules."

- Wiegers, Karl. Software Requirements Specification Template, Process Impact, 1999 (structural reference).
- IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998.

2. Overall Description

2.1 Product Perspective

The LAN-Based Secure Chatting Application is a new, self-contained system; it does not replace or integrate with any existing institutional system. It is composed of three cooperating components: a Java client, a Java application server, and a PostgreSQL relational database. The server is the single coordination point: clients never communicate with each other, the database, or the PDS module directly.

2.2 Product Functions

- User registration and login, with online/offline status.
- One-to-one messaging between authenticated users.
- Spam / potentially misleading message screening via a Python Logistic Regression model, presented to the user as a probability and label, never as a fact-check.
- A supporting Linear Regression component estimating a measurable system metric, evaluated with MAE/RMSE.

2.3 User Classes and Characteristics

User Class	Description
Registered User	Any person who has registered an account on the server. Expected to have basic computer literacy (installing/running a Java desktop client on a lab or office machine). This is the only user class in Release 1.0 — no administrator role is planned; if one becomes necessary it will be introduced as a future enhancement (Section 15).

Proposed assumption: all registered users are treated equally with respect to system privileges in Release 1.0; there is no moderation or administrative override.

2.4 Operating Environment

- OE-1: The Java client shall run on any desktop operating system (Windows, Linux, macOS) with a compatible Java Runtime Environment installed.
- OE-2: The Java application server shall run on a machine connected to the same LAN as its clients, with a compatible Java Runtime Environment and network connectivity to the PostgreSQL instance.
- OE-3: The system shall use PostgreSQL as its relational database.
- OE-4: The Python PDS module shall run in an environment with the Python interpreter and the data-science libraries required for Logistic Regression and Linear Regression (exact library versions to be finalized during implementation).
- OE-5: All components (client, server, database, PDS module) are expected to operate within a single LAN; no public Internet connectivity is required or assumed.

- OE-6: For Release 1.0, the Java Server, PostgreSQL, and Python PDS module are deployed together on a single Server Machine (see Section 17, Deployment Model). Running PostgreSQL on a separate, LAN-reachable host is architecturally possible but is not the default configuration and is not separately tested in Release 1.0. (Added by the Requirements Consistency Audit, Section 20, AUD-10, to reconcile this section with the Deployment Model.)

2.5 Design and Implementation Constraints

- CO-1: The client and server shall be implemented in Java, demonstrating object-oriented (PSOOP) design.
- CO-2: Communication between client and server shall use TCP/IP sockets.
- CO-3: The database shall be PostgreSQL; no other DBMS shall be substituted.
- CO-4: The classification component shall use Logistic Regression, and the supporting regression component shall use Linear Regression — no deep-learning, transformer, or generative-AI model shall be substituted.
- CO-5: Java-Python communication shall use a compact JSON request/response payload. The exact transport mechanism (e.g., subprocess, socket, HTTP) is to be finalized during implementation.
- CO-6: The system shall not introduce technologies such as MongoDB, Firebase, AWS, React, Node.js, Docker, or Kubernetes, except where explicitly listed as a future enhancement (Section 15).
- CO-7: The system is a monolithic LAN deployment for Release 1.0; multi-LAN federation and cloud deployment are out of scope.

2.6 Assumptions and Dependencies

- AS-1: All client and server machines are connected to the same LAN and can reach one another over TCP/IP.
- AS-2: Users are assumed to be within a trusted environment (classroom, lab, office); the threat model addresses accidental misuse and basic attacks, not a fully hostile network.
- DE-3: The exact password-hashing algorithm is to be finalized during implementation.

2.7 System Boundaries

The system boundary encloses the Java client application, the Java application server, and the PostgreSQL database, all operating within a single LAN. The LAN infrastructure itself (routers, switches, DHCP) and the operating systems of the host machines are outside the system boundary and are treated as part of the operating environment.

3. System Architecture

3.1 Architecture Overview

The system uses a layered client-server architecture with four cooperating components: the Java Client, the Java Server, the PostgreSQL database, and the Python PDS module. The server is the sole point of contact for every other component; clients never connect directly to the database or the PDS module.

Diagram specification (Component/Deployment view) — to be rendered in UML/Draw.io: Java Client (UI, Chat Module, Auth Module, File Module, Network Module) --TLS/TCP--> Java Server (ConnectionManager, AuthenticationManager, MessageRouter, GroupManager, FileManager, DatabaseManager, PDSService) --JDBC--> PostgreSQL; Java Server --JSON--> Python PDS Module (Logistic Regression, Linear Regression).

3.2 Client Architecture

- UI component: presents login/registration, contact list, conversation view.
- Chat Module: composes, sends, and displays one-to-one messages.
- Auth Module: collects credentials and manages the client-side session state.
- Network Module: manages the TCP connection to the server.

3.3 Server Architecture

Component names below are used consistently throughout this document (architecture, data-flow diagrams, class diagram, and traceability matrix) to avoid the naming drift identified in the Requirements Consistency Audit, Section 20 (AUD-09).

- ConnectionManager: accepts and manages concurrent client TCP connections.
- AuthenticationManager: verifies credentials and establishes sessions.
- MessageRouter: delivers one-to-one messages to the correct recipient connection.
- DatabaseManager: encapsulates all PostgreSQL interaction (prepared statements, transactions); also referred to as the DB Access Layer.
- PDSService: forwards message text to the Python module and relays the classification result back to the MessageRouter.

3.4 Database Layer

PostgreSQL provides persistent, relational storage for users, conversations, and messages. See Section 7 (Database Requirements) for the full schema.

3.5 Network Architecture

The application is explicitly LAN-based and is not designed for Internet-scale deployment. Clients connect to the configured server address within the local network.

- The client uses a configurable, fixed server IP address and port supplied by the user or configuration file.

3.6 Data Flow

End-to-end workflow for a single message:

- 1. CONNECT — the client connects to the configured LAN server address.
- 2. AUTHENTICATE — the user supplies credentials; the server verifies them and establishes a session.
- 3. SEND — the authenticated client transmits a message over the established connection.
- 4. PERSIST — the server writes the message and its metadata to PostgreSQL.

- 5. DELIVER — the server routes the message to the intended recipient and returns a delivery status to the sender.

3.7 Python PDS Layer

The Python PDS module implements two data-science components: a Logistic Regression classifier for spam/potentially-misleading message screening, and a Linear Regression model for estimating a supporting system metric. See Section 9 for details.

3.8 Java-Python Integration

The Java server and Python PDS module exchange compact JSON payloads. A representative request contains the message text; a representative response contains a probability and a classification label, e.g. {"probability": 0.91, "label": "potential_spam"}. The specific integration transport (subprocess invocation, local socket, or HTTP call) is an implementation decision to be finalized during development; this SRS specifies only the required request/response behavior.

4. External Interface Requirements

4.1 User Interface

- UI-1: The client shall provide a Login/Register screen collecting username and password, with separate actions for logging in and registering a new account.
- UI-2: The Main Chat Screen shall show a contact/user list with online status, a conversation list, a message display area, a message input field, and a send action.
- UI-3: The Group Chat screen shall show group information, a member list, the group's message history, and a message composer.
- UI-4: When the PDS module classifies a message as potential spam or potentially misleading, the client shall display an indicator in the form "Potential Spam — Probability: XX%" or "Potentially Misleading — Probability: XX%" adjacent to the affected message.
- UI-5: The PDS indicator shall not be worded in a way that implies the message's factual truth has been verified; it shall be presented strictly as a model prediction/probability.
- UI-6: Form validation errors (e.g., empty message, duplicate username) shall be displayed inline in plain language.

4.2 Hardware Interfaces

No special hardware interfaces are required. The system runs on standard desktop/laptop computers equipped with a network interface connected to the LAN. No hardware such as barcode scanners, biometric readers, or specialized peripherals is in scope.

Interface	Purpose
SI-1: Client ↔ Server (TCP)	The client shall establish an authenticated TCP connection to the server and shall exchange messages and status updates over that connection.

Interface	Purpose
SI-2: Server ↔ PostgreSQL (JDBC)	The server shall use a JDBC connection to PostgreSQL for all persistence operations, using prepared statements for parameterized queries.
SI-3: Server ↔ Python PDS Module	The server shall send message text to the PDS module as a JSON request and shall receive a JSON response containing a classification probability and label (and, where applicable, a Linear Regression estimate).

4.3 Communication Interfaces

- CI-1: All client-server communication shall occur over TCP/IP within the LAN.
- CI-2: Client-server communication shall be protected using an encrypted transport mechanism (TLS over the TCP socket is the proposed approach; see Section 8).
- CI-3: Server-to-PDS communication shall use compact JSON payloads exchanged over the mechanism finalized during implementation (Section 3.6).

5. Functional Requirements

Each functional requirement below is uniquely identified, testable, and unambiguous, and is grouped by module as required by the project scope. Requirements are written using "the system shall..." phrasing wherever possible.

5.1 User Registration and Management (Module A)

ID	Requirement
FR-01	The system shall allow a new user to register an account by providing a unique username and a password.
FR-02	The system shall reject registration if the requested username is already in use.
FR-03	The system shall allow a registered user to log in using their username and password.
FR-04	The system shall reject login attempts with invalid credentials and shall display a generic failure message that does not reveal whether the username or password was incorrect.
FR-05	The system shall allow an authenticated user to log out, ending their session and marking them offline.
FR-06	The system shall track each registered user's online/offline status.

5.2 LAN Connectivity (Module B)

ID	Requirement
FR-07	The client shall connect to the server using a configured/fixed server address.
FR-08	The system shall support multiple clients connecting to the server concurrently.
FR-09	The client shall detect a lost server connection and shall notify the user rather than silently failing.
FR-10	The client shall provide a way to retry connecting to the server after a connection failure.

5.3 Authentication (Module C)

ID	Requirement
FR-11	The system shall verify user credentials against securely stored (hashed) values before granting access to protected functionality.
FR-12	The system shall establish an authenticated session upon successful login and shall associate all subsequent actions on that connection with the logged-in user.
FR-13	The system shall deny access to messaging and history functionality to any connection that has not completed authentication.
FR-14	The system shall terminate a session on logout or on connection loss and shall require re-authentication for further access.

5.4 One-to-One Chat (Module D)

ID	Requirement
FR-15	The system shall allow an authenticated user to start a one-to-one conversation with another registered user.
FR-16	The system shall allow an authenticated user to send a text message to another authenticated user through the server.
FR-17	The system shall route a one-to-one message from the sender to the intended recipient if the recipient is currently connected.
FR-18	The system shall persist every one-to-one message, associating it with the sender, recipient/conversation, and a timestamp.
FR-19	The system shall allow a user to retrieve the message history of a one-to-one conversation they participate in.
FR-20	The system shall return a delivery/status acknowledgement to the sender after a message is processed by the server.

5.5 Message Persistence and History (Module F)

ID	Requirement
FR-21	The system shall store every sent one-to-one message in PostgreSQL with sender, conversation, timestamp, and message text.
FR-22	The system shall ensure every stored message is associated with exactly one conversation and one sender.

5.6 Error Handling (Module K)

ID	Requirement
FR-23	The system shall display a clear error message when login fails, registration is attempted with a duplicate username, or the server is unreachable.
FR-24	The system shall handle client disconnection gracefully on the server side, releasing associated resources without affecting other connected clients.
FR-25	The system shall handle a database-unavailable condition by informing the user that persistence is temporarily unavailable, rather than crashing.
FR-26	The system shall reject an empty message and shall inform the sender rather than transmitting or persisting it.

5.7 PDS Message Classification (Module I)

ID	Requirement
FR-27	The system shall submit the text of each sent message to the Python PDS module for classification before or concurrently with delivery.
FR-28	The PDS module shall extract features from the message text and compute a probability using Logistic Regression (see Section 9).
FR-29	The system shall compare the computed probability to a configurable threshold to assign a classification label (e.g., Normal, Potential Spam, Potentially Misleading).
FR-30	The system shall return the classification probability and label to the Java application and shall associate it with the corresponding stored message.
FR-31	The client shall display the PDS classification to the recipient (and, optionally, the sender) as a probability/label, never as a verified fact.
FR-32	If the PDS module is unavailable, the system shall still deliver and persist the message, marking the classification as unavailable rather than blocking the message (see Section 12).

5.8 Supporting Analytics / Linear Regression (Module J)

ID	Requirement
FR-33	The system shall collect measurable operational observations (e.g., timestamps needed to compute message delivery latency) suitable as input to the Linear Regression component.
FR-34	The PDS module shall provide a Linear Regression estimate for the chosen supporting system metric, with the exact target metric finalized during implementation (see Section 9.9).
FR-35	The system shall support evaluation of the Linear Regression model using MAE and RMSE, computed during testing rather than fabricated.

5.9 Error Handling (Module K)

ID	Requirement
FR-36	The system shall display a clear error message when login fails, registration is attempted with a duplicate username, or the server is unreachable.
FR-37	The system shall handle client disconnection gracefully on the server side, releasing associated resources without affecting other connected clients.
FR-38	The system shall handle a database-unavailable condition by informing the user that persistence is temporarily unavailable, rather than crashing.
FR-39	The system shall log a malformed-JSON or unreachable-PDS-module condition. Resulting message delivery behavior is governed by FR-39, which is the single source of truth for this fallback, so the two requirements cannot drift out of sync as either is updated.
FR-40	The system shall reject an empty message and shall inform the sender rather than transmitting or persisting it.

6. Non-Functional Requirements

6.1 Performance

ID	Requirement
NFR-01	The system shall route a one-to-one message to an online recipient within a measurable latency target under defined LAN test conditions; the target value shall be established during performance testing.

6.2 Security

ID	Requirement
NFR-02	The system shall require successful authentication before granting access to messaging or history functionality.
NFR-03	The system shall store passwords using an appropriate password-hashing algorithm; passwords shall never be stored or logged in plaintext.
NFR-04	The system shall use prepared/parameterized statements for all database queries to reduce injection risk.
NFR-05	The system shall validate all client input before processing or persisting it.

6.3 Reliability

ID	Requirement
NFR-06	The system shall handle a client disconnection without affecting other connected clients' sessions.
NFR-07	The system shall log server-side errors without crashing the server process.
NFR-08	The system shall handle a temporary database outage by informing affected users rather than losing in-flight messages where practical.

6.4 Scalability

The system is intended for a small/academic LAN environment (a single classroom, lab, or office), not Internet-scale deployment. The exact number of concurrently supported clients shall be established during performance testing rather than assumed.

6.5 Usability

ID	Requirement
NFR-09	A first-time user shall be able to locate the login form, contact list, and message input without external help, as verified by a usability walkthrough with at least one test user.
NFR-10	The client shall clearly indicate each contact's online/offline status.
NFR-11	The client shall clearly present message delivery status without technical jargon.

6.6 Maintainability

ID	Requirement
NFR-12	The Java implementation shall separate networking, database access, and UI logic into distinct modules/classes.

6.7 Portability

The Java client and server shall run on any desktop operating system with a compatible Java Runtime Environment (Windows, Linux, macOS).

6.8 Data Integrity

ID	Requirement
NFR-13	The system shall ensure every stored message is correctly associated with its sender and conversation.
NFR-14	The system shall record accurate timestamps for all messages and file transfers.
NFR-15	The system shall use transactional database operations where multiple related writes must succeed or fail together (e.g., message insert plus classification update).

7. Database Requirements

7.1 Database Overview

PostgreSQL is the system of record for users, conversations, and messages. The schema below is a recommended starting point consistent with the project's conceptual structure; it may be refined during implementation provided the entities and relationships below remain intact.

7.2 Entities

- User — a registered account (identity, credential hash, status, creation time).
- Conversation — a one-to-one conversation container.
- Message — a single sent message, linked to its conversation and sender.

7.3 Relationships

- A User may belong to many Conversations, and a Conversation has many Users.
- A Conversation has many Messages; each Message belongs to exactly one Conversation and one sending User (one-to-many).

7.4 Schema (Conceptual)

Table	Key Columns
USERS	user_id (PK), username (unique), password_hash, status, created_at
CONVERSATIONS	conversation_id (PK), created_at
MESSAGES	message_id (PK), conversation_id (FK), sender_id (FK), message_text, timestamp

7.5 Constraints

- username in USERS shall be unique and not null.
- password_hash shall never store a plaintext password.
- Every row in MESSAGES shall reference a valid conversation_id and sender_id (foreign-key constraints).

7.6 Indexing

To mitigate the database-bottleneck risk identified in Section 14 (Risk 2), the following indexes are recommended design choices (not fixed requirements):

- Index on MESSAGES(conversation_id) to accelerate history retrieval.
- Index/unique constraint on USERS(username) to accelerate login lookups.

7.7 Transactions

Recommended design choice: message insertion shall be wrapped in a database transaction so that a partially completed write cannot leave an incomplete message.

7.8 Data Security

- Database credentials shall not be hard-coded in client-distributed artifacts; they shall be held only by the server.
- All database access shall go through the server's DatabaseManager component using prepared statements (see Section 8).

8. Security Requirements

Security is a core project objective, with basic authentication, password protection, database security, and input validation specified for the system.

8.1 Authentication

- SEC-01: The system shall require a valid username and password before granting a session.
- SEC-02: The system shall deny access to messaging and history operations on any connection that has not authenticated.

8.2 Authorization

- SEC-03: A user shall be able to access only conversations associated with their own account.

8.3 Password Protection (Credential Security)

- SEC-04: Passwords shall be stored using an appropriate one-way hashing algorithm (e.g., bcrypt or an equivalent salted hash); the exact algorithm is to be finalized during implementation.
- SEC-05: Passwords shall never be stored, logged, or displayed in plaintext at any point in the system. A password exists in plaintext only transiently in memory while it is entered by the user and while it is hashed for verification or storage.

8.4 Database Security

- SEC-07: All SQL statements shall use parameterized/prepared statements to mitigate injection risk.
- SEC-08: Database credentials shall be restricted to the DatabaseManager component and shall not be embedded in the client.

8.5 Input Validation

- SEC-09: The system shall validate message input before accepting it, rejecting invalid input with a clear error.

8.6 Session Management

- SEC-10: A session shall be invalidated on logout or on loss of the underlying TCP connection, requiring re-authentication for further access.

8.7 Threat Considerations and Explicit Limitations

The system is designed for a trusted LAN environment and does not claim protection against a fully hostile network or a compromised server administrator. The following claims are explicitly out of scope and must not be made about this system:

- The system is NOT described as completely secure, unhackable, or military-grade.
- The system does NOT provide full end-to-end encryption; message content is stored in the PostgreSQL database in a form accessible to the server component.
- The system does NOT provide anonymity; all messages are associated with an authenticated identity.

Data Science / PDS Requirements

Logistic Regression

The system uses Logistic Regression to screen messages for patterns associated with spam or potentially misleading content. The model conceptually computes p(y=1|x) = 1 / (1 + e^-z), where x is a vector of extracted message features and z is a linear combination of those features. This is a classical statistical classification technique, not deep learning or generative AI.

Input Features

Potential text-derived features include TF-IDF or term-frequency values, message length, presence of suspicious terms, and repeated-pattern indicators. The final feature-extraction technique will be selected during implementation and model evaluation; no specific NLP technique is presented here as already implemented.

Classification Process

- PDS-01: The system shall extract features from each submitted message text.
- PDS-02: The system shall compute a classification probability using the trained Logistic Regression model.
- PDS-03: The system shall compare the probability to a configurable threshold to assign a label.

Probability Output

The PDS module's output shall always include a numeric probability (e.g., P(spam) = 0.87) alongside its label, so that the result is presented as a model prediction rather than a binary assertion.

Threshold

The classification threshold shall be configurable rather than hard-coded; its final value shall be determined during model evaluation (recommended starting point: 0.5, subject to change based on precision/recall analysis).

Model Evaluation

The Logistic Regression model shall be evaluated using accuracy, precision, recall, F1-score, and a confusion matrix, and via threshold analysis. These figures shall be obtained through actual testing and shall not be fabricated or assumed in advance.

False Positives / False Negatives

The classifier may produce false positives (flagging a legitimate message) and false negatives (missing an actual spam/misleading message). This is an accepted limitation to be quantified during evaluation, not eliminated.

Limitations

- The classifier does not determine whether a statement is factually true or false; it detects patterns learned from training data.
- A result such as "Potentially Misleading — Confidence: 78%" is a model prediction, not proof that the message is false.
- The system shall never present PDS output as a fact-check or truth-verification result.

Linear Regression

A Linear Regression component estimates a measurable, supporting system metric (candidate metrics include message delivery latency, database response time, or processing time). This is a supporting component, not the primary purpose of the application. The exact target metric shall be finalized during implementation.

MAE / RMSE

The Linear Regression component shall be evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), computed from actual test data during the evaluation phase, and reported honestly rather than assumed.

9. Use Cases

UC-01 — Register

Field	Detail
Actor	Unregistered User
Description	A visitor creates a new account.
Preconditions	The user is not already registered.
Trigger	User submits a username and password.
Main Flow	1. User provides username/password. 2. System checks uniqueness. 3. System creates the account.
Alternative Flow	None.
Exception Flow	Username already exists → system rejects with a clear message.
Postconditions	A new user account exists in the database.

UC-02 — Login

Field	Detail
Actor	Registered User
Description	A registered user authenticates to the system.
Preconditions	The user has a registered account and is connected to the server.
Trigger	User submits credentials.
Main Flow	1. User provides username/password. 2. Server verifies against stored hash. 3. Server establishes a session and marks the user online.
Alternative Flow	None.
Exception Flow	Invalid credentials → generic failure message; no session created.
Postconditions	The user has an authenticated session and is marked online.

UC-03 — Connect to LAN Server

Field	Detail
Actor	Client Application
Description	The client establishes a network connection to the server.
Preconditions	The server is running and reachable on the LAN.
Trigger	The client application is started.
Main Flow	1. Client uses the configured server address. 2. Client opens a TCP connection.
Alternative Flow	None.
Exception Flow	Server unreachable → client displays a connection-failure message and offers retry.
Postconditions	A network connection exists between client and server, ready for authentication.

UC-04 — View Online Users

Field	Detail
Actor	Registered User
Description	An authenticated user views which contacts are currently online.
Preconditions	The user is authenticated.
Trigger	User opens the contact list.
Main Flow	1. Client requests current status list from server. 2. Server returns online/offline status for each contact.
Alternative Flow	None.
Exception Flow	Server unreachable → client shows last-known status with an indication that it may be stale.
Postconditions	The user sees an up-to-date contact status list.

UC-05 — Send Private Message

Field	Detail
Actor	Registered User
Description	An authenticated user sends a one-to-one message.

Field	Detail
Preconditions	Both sender and recipient are registered; sender is authenticated.
Trigger	User composes and sends a message to a contact.
Main Flow	1. User composes message. 2. Client sends it to server. 3. Server persists it and routes it to the recipient if online. 4. Server returns delivery status.
Alternative Flow	Recipient offline → the message is still persisted per FR-18 and becomes visible to the recipient through ordinary history retrieval (FR-19/UC-10) the next time they connect; no separate offline push-notification queue is implied or required.
Exception Flow	Empty message → rejected before transmission. Server unreachable → client reports failure.
Postconditions	The message is persisted and, where possible, delivered.

UC-06 — Receive Private Message

Field	Detail
Actor	Registered User
Description	An authenticated user receives a one-to-one message.
Preconditions	Recipient is authenticated and connected.
Trigger	Server routes an incoming message to this client.
Main Flow	1. Server pushes message to recipient's connection. 2. Client displays it.
Alternative Flow	None.
Exception Flow	Connection drops mid-delivery → message remains persisted and retrievable on reconnection.
Postconditions	The message is displayed to the recipient.

UC-07 — View Chat History

Field	Detail
Actor	Registered User
Description	A user retrieves past messages of a conversation they participate in.
Preconditions	The user is authenticated and a participant in the conversation.
Trigger	User opens a conversation.
Main Flow	1. Client requests chat history. 2. Server queries PostgreSQL and returns the history.
Alternative Flow	None.
Exception Flow	Database unavailable → server informs the client that history is temporarily unavailable.
Postconditions	The requested page of history is displayed.

UC-08 — Logout

Field	Detail
Actor	Registered User
Description	An authenticated user ends their session.

Field	Detail
Preconditions	The user is authenticated.
Trigger	User requests to log out.
Main Flow	1. Client sends logout request. 2. Server invalidates the session and marks the user offline.
Alternative Flow	None.
Exception Flow	Connection already lost → server treats the disconnection itself as an implicit logout.
Postconditions	The session is terminated and the user is marked offline.

10. System Models

Diagrams are provided here as precise specifications rather than rendered images; they can be converted directly into UML/Draw.io/PlantUML diagrams during implementation.

10.1 Use Case Diagram

Aspect	Detail
Purpose	Show all actor-to-use-case relationships for the system.
Actors / Components	Registered User (only actor in Release 1.0); Server treated as system boundary.
Relationships	Registered User associates with UC-01 through UC-13 (Section 10).
What it represents	The complete set of user-facing capabilities of the system.

10.2 Context Diagram

Aspect	Detail
Purpose	Show the system as a single node with its external interfaces.
Actors / Components	Java Client, Java Server (the system), PostgreSQL.
Relationships	Client ↔ Server (TCP); Server ↔ PostgreSQL (JDBC).
What it represents	The system boundary and its three external interfaces.

10.3 DFD Level 0

Aspect	Detail
Purpose	Show the system as a single process with data flows to/from external entities.
Actors / Components	External entities: User; Data store: Message Store (PostgreSQL).
Relationships	User → [Chat System] → User; [Chat System] ↔ Message Store.
What it represents	Top-level data flow through the system.

10.4 DFD Level 1

Aspect	Detail
Purpose	Decompose the system process into Authentication, Message Routing, and Persistence sub-processes.
Actors / Components	AuthenticationManager, MessageRouter, DatabaseManager.
Relationships	User credentials → AuthenticationManager; authenticated message → MessageRouter → DatabaseManager → recipient.
What it represents	How a message moves through the server's internal sub-processes.

10.5 Class Diagram

Aspect	Detail
Purpose	Show the primary object-oriented classes and their relationships.
Actors / Components	User, Message, Conversation, Client, Server, ClientHandler, AuthenticationManager, MessageRouter, DatabaseManager.
Relationships	Server has many ClientHandlers; ClientHandler uses AuthenticationManager and MessageRouter; MessageRouter uses DatabaseManager; Conversation has many Messages.
What it represents	The conceptual object-oriented structure (class names are illustrative, not finalized).

10.6 Sequence Diagram — Send Message

Aspect	Detail
Purpose	Show the message flow from sender to recipient including persistence.
Actors / Components	Client A, Server, PostgreSQL, Client B.
Relationships	Client A → Server: send(message); Server → PostgreSQL: persist(message); Server → Client B: deliver(message); Server → Client A: status.
What it represents	The end-to-end runtime interaction for UC-05/UC-06.

10.7 Activity Diagram — Login and Send

Aspect	Detail
Purpose	Show the control flow from connection through authentication to message delivery.
Actors / Components	Client, Server.
Relationships	Connect → Authenticate → [success?] → Compose Message → Send → Persist → Classify → Deliver.
What it represents	The end-to-end workflow described in Section 3.8.

10.8 ER Diagram

Aspect	Detail
Purpose	Show entities and relationships for the database.

Aspect	Detail
Actors / Components	User, Conversation, Message.
Relationships	Conversation (1) — (M) Message; Message (M) — (1) User.
What it represents	The relational schema described in Section 7.

10.9 Deployment Diagram

Aspect	Detail
Purpose	Show the physical machines and the components deployed on each.
Actors / Components	Server Machine (Java Server, PostgreSQL), Client Machines (Java Client) × N.
Relationships	Client Machines —TCP— Server Machine; Server Machine internal: Java Server —JDBC— PostgreSQL.
What it represents	The physical deployment topology described in Section 13.

10.10 Component Diagram

Aspect	Detail
Purpose	Show the software components and their dependencies.
Actors / Components	UI, Chat Module, Auth Module, Network Module (client); ConnectionManager, AuthenticationManager, MessageRouter, DatabaseManager (server).
Relationships	As described in Sections 3.2 and 3.3.
What it represents	The modular software architecture.

10.11 Network Topology (DSGT Mapping)

Aspect	Detail
Purpose	Model the LAN as a graph for subject-mapping purposes (Discrete Structures and Graph Theory).
Actors / Components	Vertices: each Client and the Server. Edges: active TCP connections.
Relationships	Client A — Server, Client B — Server, Client C — Server (star topology centered on the server).
What it represents	Connectivity, reachability, and availability of the LAN communication paths; not intended as a complex graph-algorithm module.

11. Testing Requirements

11.1 Unit Testing

- Authentication functions (credential verification, session creation).
- Message validation (empty-message rejection, length limits).

11.2 Integration Testing

- Java Client ↔ Java Server (connection, authentication, message exchange).
- Java Server ↔ PostgreSQL (persistence and retrieval).

11.3 System Testing

End-to-end workflow tests shall cover: Login → Connect → Send message → Persist → PDS analysis → Deliver → Retrieve history, for both one-to-one and group conversations.

11.4 Security Testing

- Invalid login attempts are rejected with a generic error.
- Unauthenticated connections cannot access protected operations.
- Invalid or malformed input is rejected without crashing the server.
- SQL injection resistance of all database-facing inputs.
- Unauthorized access to another user's conversation is denied.

11.5 Performance Testing

- Message delivery latency under varying numbers of concurrent clients.
- Database response time for history retrieval.
- PDS inference latency.
- Overall system response time under normal LAN conditions.

11.6 Model Evaluation

Logistic Regression: accuracy, precision, recall, F1-score, confusion matrix, and threshold analysis, computed from actual evaluation data. Linear Regression: MAE and RMSE, computed from actual evaluation data. No numerical results are asserted in this document; they are to be obtained during testing.

12. Risks and Mitigations

Risk	Mitigation
Risk 1 — LAN discovery failure	Provide a fixed/configurable server IP as fallback; treat automatic discovery as optional; test across multiple hosts.
Risk 2 — Database bottleneck	Indexing on conversation_id, sender_id, timestamp, username; prepared statements; pagination; efficient queries.
Risk 3 — Overstated security claims	Clearly distinguish transport, credential, and storage security (Section 8); avoid absolute security claims.
Risk 4 — Classifier false positives/negatives	Present results as probabilities; use a configurable threshold; evaluate with precision/recall; allow future user feedback.
Risk 5 — Weak Linear Regression model	Use only for a measurable, short-term supporting metric; evaluate with MAE/RMSE; report honestly.

Risk	Mitigation
Risk 6 — Python-Java integration latency	Favor local inference; use compact JSON; avoid unnecessary network round trips.

13. Representative Test Cases

Test cases marked (new) were added to close traceability gaps identified during requirements review.

Test ID	Requirement	Scenario	Input	Expected Output	Result
TC-01	FR-04	Login with invalid password	Valid username, wrong password	Generic “incorrect username or password” message; no session created	To be executed
TC-01a (new)	FR-03	Login with valid credentials	Registered username and correct password	Session established; user marked online	To be executed
TC-02 (new)	FR-01/FR-02	Register with a duplicate username	A username already in use	Registration rejected with a clear duplicate-username error	To be executed
TC-03 (new)	FR-05/FR-06	Logout and status visibility	Authenticated user logs out; a non-contact user checks status	User marked offline on logout; online/offline status is displayed in the contact list.	To be executed
TC-04	FR-17	Send message to online recipient	Authenticated sender, online recipient, text message	Recipient receives the message; sender receives delivery status	To be executed
TC-05	FR-26	Persist and retrieve message	A sent message	Message retrievable from PostgreSQL after session ends	To be executed
TC-06 (new)	FR-43–FR-47	General error conditions	Empty message; DB unavailable	Each condition produces a clear error/fallback without a server crash.	To be executed
TC-07 (new)	NFR-12	Usability walkthrough	A first-time test user given no instructions	User locates login, contact list, and message input unaided	To be executed

14. Deployment Model

The system is deployed entirely within a single LAN. No cloud infrastructure is assumed.

Machine	Responsibilities
Server Machine	Runs the Java Server and PostgreSQL database. Must remain reachable to all client machines over the LAN.

Machine	Responsibilities
Client Machines	Run the Java Client application. Connect to the Server Machine using a configured address.

Diagram specification: LAN/Wi-Fi Network — Server Machine (Java Server, PostgreSQL) — Client Machines (Java Client) × N, all connected to the same LAN segment.

15. Explicit Project Limitations

- LAN dependency: the system is intended for users connected to the same local network, not the public Internet.
- Limited scale: this is an academic/small-network system, not an Internet-scale platform; concurrency targets are established during testing, not assumed.
- Security limitation: the system provides basic authentication and password protection but does not claim complete security.

16. Future Enhancements

The following are explicitly future scope and are not requirements of Release 1.0:

- Administrator role with moderation controls.
- Password/account recovery.
- Deployment across multiple LANs or larger networks.
- Voice and video communication.
- Mobile application support.
- Advanced security such as end-to-end encryption.

Appendix A: Team Responsibilities and Timeline

B.1 Team Responsibilities

Member	Responsibility
Aryan Patil	Networking; Java server/client implementation.
Malhar Nimbalkar	DBMS; backend / persistence layer.
Amogh Sirsirkar	UI implementation.

B.2 Indicative Project Timeline (14 Weeks)

Activity	Weeks
Requirements + literature review	1–4
UI + LAN socket prototype	4–7
PostgreSQL schema + CRUD	5–8
Authentication + security	8–11
Chat messaging	9–12
Integration + testing	13–14

Activity	Weeks
Performance + security testing	14
Report + presentation + demo	14

Appendix C: To Be Determined List

The following implementation details are intentionally left open and are tagged for closure during implementation rather than being invented in this document:

- TBD-3: Exact password-hashing algorithm (recommended: bcrypt or equivalent).
- TBD-8: Exact number of concurrently supported clients (to be established during performance testing).
- TBD-9: Exact Java UI framework/toolkit for the client.
- TBD-10: Exact Java-Python integration transport (subprocess, local socket, or HTTP).